# ASSIGNMENT-03

## Question 1

**What is Test Planning? Explain the main steps involved in the test planning process.**

### Answer

Test planning is the activity of deciding in advance what will be tested, how it will be tested, who will test it, with which tools and in what time. It is the process of defining the scope, approach, resources, schedule and risks of the testing effort, and its result is a document called the Test Plan. Good planning makes testing organised and measurable, helps in estimating cost and time, and avoids last-minute confusion.

### Main steps in the test planning process

- **1. Study the product and requirements:** the test team reads the SRS, design documents and user manuals to understand what the software must do, its users and its risky areas.
- **2. Define the scope of testing:** decide which features and modules will be tested (in scope) and which will not be tested (out of scope), with the reasons.
- **3. Set the test objectives and exit criteria:** decide what the testing must achieve, for example 100% of high-priority test cases executed and no open critical defects.
- **4. Decide the test strategy and approach:** select the levels (unit, integration, system, acceptance), types (functional, performance, security) and techniques (black-box, white-box) of testing, and whether to use manual or automation testing.
- **5. Identify the test environment and resources:** list the people, skills, hardware, software, tools and test data that are required.
- **6. Estimate effort and prepare the schedule:** estimate the time and cost for test design, execution and re-testing, and fix milestones and deadlines.
- **7. Assign roles and responsibilities:** decide who writes test cases, who executes them, who reports defects and who approves.
- **8. Risk analysis and contingency planning:** identify risks such as late delivery, unstable builds, shortage of testers and plan how to handle them.
- **9. Define the test deliverables and reporting:** decide which documents (test cases, defect reports, summary report) will be produced and how progress will be reported.
- **10. Review and approve the plan:** the test plan is reviewed by the stakeholders, approved, and updated whenever the project changes.

## Question 2

**What is a Test Strategy? Explain its importance in software testing.**

### Answer

A Test Strategy is a high-level document that describes the general approach to testing for a whole organisation or a product. It explains how testing will be done in general: which test levels and types will be used, which techniques and tools will be followed, how defects will be managed and how quality will be measured. It is usually prepared by the test manager or test lead in the early phase of the project and does not change frequently.

### Contents of a test strategy

- Scope and objectives of testing
- Test levels and test types to be used
- Test techniques (black-box, white-box, experience-based)
- Test environment and tools, including automation approach
- Defect management and reporting procedure
- Entry and exit criteria
- Risk and mitigation approach
- Metrics to be collected and configuration management

### Importance of a test strategy

- **Gives direction:** all testers follow the same approach, so the work is consistent.
- **Saves time and cost:** the right type of testing is chosen in advance and unnecessary testing is avoided.
- **Improves quality:** high-risk areas get more attention and the product is tested systematically.
- **Helps in planning:** the test plan, estimation and resource allocation are based on the strategy.
- **Better communication:** developers, managers and customers understand how the product will be tested.
- **Risk reduction:** risks are identified early and handled properly.
- **Reusability:** the same strategy can be reused for similar projects.

## Question 3

**What is a Test Plan? Explain the major contents of a Test Plan.**

### Answer

A Test Plan is a detailed document that describes the scope, approach, resources and schedule of the testing activities for a particular project or release. It tells what to test, how to test, who will test, when to test and what the criteria for completion are. IEEE 829 gives a standard format for test plan documents. The test plan is a project-level document and acts as an agreement between the testing team, the development team and the management.

### Major contents of a test plan

- **Test plan identifier:** unique name, number and version of the document.
- **Introduction:** purpose of the plan and brief description of the project and references to other documents.
- **Test items:** the software modules, versions or builds to be tested.
- **Features to be tested:** the functions and non-functional characteristics that are in scope.
- **Features not to be tested:** items excluded, with reasons.
- **Approach:** the test levels, types, techniques and tools used.
- **Item pass / fail criteria:** the rule to decide whether a test item has passed.
- **Suspension and resumption criteria:** conditions when testing is stopped (for example a blocking defect) and restarted.
- **Test deliverables:** test cases, test data, defect reports, test summary report.
- **Testing tasks:** list of activities, with dependencies.
- **Environment needs:** hardware, software, network, tools and test data.
- **Responsibilities and staffing:** roles of each team member and training needs.

- **Schedule:** milestones and deadlines of the testing tasks.
- **Risks and contingencies:** possible problems and the plan to handle them.
- **Approvals:** the people who review and sign the plan.

# Question 4

**Differentiate between a Test Strategy and a Test Plan.**

## Answer

| Basis | Test Strategy | Test Plan |
|---|---|---|
| Meaning | High-level document describing the general approach to testing. | Detailed document describing what, how, who and when of testing a particular project. |
| Level | Organisation or product level. | Project or release level. |
| Prepared by | Test manager / senior QA. | Test lead / test manager. |
| Scope | General principles that are applicable to many projects. | Specific to one project. |
| Change | Static; rarely changes. | Dynamic; changes as the project changes. |
| Details | Test levels, types, techniques, tools, standards. | Scope, items, schedule, resources, risks, entry and exit criteria. |
| Existence | Can be a part of the test plan or a separate document. | Is derived from the test strategy. |
| Timing | Prepared first, early in the project. | Prepared after the strategy, before test execution. |
| Focus | How testing will be approached. | How testing will be carried out. |
| Example | "Both manual and automation testing will be used; regression tests will be automated." | "Login module will be tested from 1 to 10 March by two testers on Chrome and Firefox." |

# Question 5

**What are the main goals and objectives of software testing? Explain.**

## Answer

Software testing is the process of executing a program with the intention of finding defects and evaluating whether it meets the specified requirements. The main goal of testing is to give confidence that the software is of good quality and fit for use. The goals and objectives are:

- **Finding defects:** the first aim is to detect errors, faults and failures as early as possible, before the software reaches the user.
- **Preventing defects:** reviewing requirements and design finds mistakes before they become defects in the code.
- **Verifying requirements:** checking that the software does what the specification says (verification).
- **Validating user needs:** checking that the software solves the real problem of the user (validation).
- **Building confidence in quality:** giving the stakeholders information about the quality level so that they can decide on release.
- **Reducing risk:** lowering the chance of failure in production, which could cause loss of money, data or reputation.
- **Reducing cost:** an early defect is cheaper to repair than a defect found after release.

- **Ensuring non-functional quality:** checking performance, security, usability, reliability and compatibility.
- **Ensuring compliance:** meeting legal, regulatory and contractual standards.
- **Providing information for decision making and improvement:** test results and metrics help to improve the process and the product.

It must be remembered that testing can show the presence of defects but cannot prove that there are no defects.

# Question 6

**What are Test Phases? Explain the different phases of software testing.**

## Answer

Test phases (the software testing life cycle, STLC) are the sequence of stages through which the testing activity is carried out, from understanding the requirements to the closing of the test cycle. Each phase has entry criteria, activities and deliverables. The common phases are:

- **1. Requirement analysis:** the testers study the requirements to find the testable items, identify the type of tests required and raise doubts about unclear requirements. Output: list of testable requirements and requirement traceability matrix (RTM).
- **2. Test planning:** the test strategy and test plan are prepared with scope, effort, resources, tools, schedule and risks. Output: test plan and effort estimate.
- **3. Test case development (test design):** test cases and test data are designed and written and reviewed. Output: test cases, test scripts and test data.
- **4. Test environment setup:** the hardware, software, network and tools needed for execution are installed and a smoke test checks that the build is ready.
- **5. Test execution:** test cases are executed, actual results are compared with expected results, defects are reported and tracked, and fixed defects are re-tested with regression testing. Output: test execution report and defect reports.
- **6. Test cycle closure:** the team evaluates the completion criteria, analyses the metrics, prepares the test summary report and records the lessons learned. Output: test closure report.

### Levels of testing (also called phases)

In another view, the test phases are the levels of testing: **unit testing** (individual components by developers), **integration testing** (interfaces between modules), **system testing** (the complete system against requirements) and **acceptance testing** (the customer checks that the system satisfies business needs), followed by regression testing during maintenance.

# Question 7

**What factors should be considered while developing a Test Strategy?**

## Answer

A test strategy must be developed carefully as it affects the whole testing effort. The following factors are considered:

- **Project requirements and scope:** the size and complexity of the application and the number of features decide the depth of testing.
- **Risk:** the areas where a failure would have a high impact (payment, security) are tested more intensely.

- **Type of application and domain:** a banking or medical product needs stricter testing than a simple website; the domain rules and regulations must be followed.
- **Test objectives and quality expectations:** the quality level expected by the customer, such as reliability and performance targets.
- **Time and budget:** a short deadline or a low budget limits what can be done and increases the need for prioritisation.
- **Resources and skills:** the number of testers, their experience and the training needed.
- **Test environment and tools:** availability of devices, servers, test data and tools, and the possibility of automation.
- **Development model:** the approach is different in Waterfall, Agile and DevOps (for example continuous testing in Agile).
- **Technology and architecture:** web, mobile, cloud or embedded technology affects the testing types required.
- **Test levels, types and techniques:** decision about which are needed and how much of each.
- **Entry and exit criteria:** conditions to start and to stop testing.
- **Defect management and reporting:** tools, severity levels and the reporting procedure.
- **Standards and compliance:** company standards, legal and security requirements.
- **Customer and stakeholder involvement:** how and when users will take part in the acceptance testing.
- **Lessons from earlier projects:** past defect data and experience.

# Question 8

**Explain the resource requirements needed for software testing.**

## Answer

Resource requirements are everything that is needed to carry out testing successfully. Identifying them in advance in the test plan avoids delays and cost overrun. The resources are of the following types:

### 1. Human resources

- **Test manager / test lead:** plans, coordinates, monitors and reports.
- **Test analysts / testers:** design test cases, execute them and report defects.
- **Automation engineers:** write and maintain automated test scripts.
- **Specialists:** performance, security and usability testers, and database or network experts when required.
- **Support staff:** developers for defect fixing, system administrators for environment, business analysts for clarifications.
- **Training:** testers must have the domain knowledge and the tool knowledge, and training should be planned if the skills are missing.

### 2. Hardware resources

- Servers, computers and mobile devices of different models
- Network equipment for load and network testing
- Different operating systems and browsers for compatibility checks

### 3. Software resources

- Operating systems, databases, web servers, browsers and other platform software
- Test management tools (for example Jira, TestLink), defect tracking tools

- Automation tools (for example Selenium), performance tools (for example JMeter) and security scanners
- Version control and build tools

### 4. Test environment and test data

- A test environment similar to the production environment, separate from development
- Test data in sufficient quantity, including valid, invalid and boundary data, preferably masked so that real personal data is not exposed

### 5. Time and budget

- Effort in person-days for test design, execution and re-testing
- Budget for tool licences, devices, training and cloud services

### 6. Documents and information

- Requirement documents, design documents, user manuals, previous defect reports and test cases for reference

## Question 9

**What is a Test Schedule? Explain the importance of scheduling in software testing.**

### Answer

A Test Schedule is a timetable of the testing activities. It shows each task, its start and end dates, its duration, dependencies, the person responsible and the milestones. It is prepared in the test planning phase, using the estimate of effort, the delivery dates of builds and the availability of resources, and it is usually part of the test plan.

### Typical items in a test schedule

- Test planning and test design
- Test environment setup
- Receiving the builds and smoke testing
- Test execution cycles (cycle 1, cycle 2, regression)
- Defect reporting, fixing and re-testing
- UAT support and release sign-off
- Test closure and report

### Importance of scheduling

- **Timely completion:** gives a clear deadline for each activity so that testing finishes before the release date.
- **Proper use of resources:** testers, tools and environments are allotted at the right time without idle time or overload.
- **Tracking and control:** actual progress can be compared with the plan and delays can be seen early.
- **Coordination:** the testers and developers know when builds and fixes are expected, so waiting time is reduced.
- **Risk management:** buffer time is planned for delayed builds and defect-fixing cycles.
- **Cost control:** avoids overtime and last-minute rush.
- **Quality:** enough time is reserved for test design, execution and regression, so testing is not cut short.

- **Communication:** the stakeholders know the expected progress and release dates.

# Question 10

**What is a Test Case? Explain the main components of a test case.**

## Answer

A test case is a set of conditions, inputs, steps and expected results that is prepared to check one particular feature or requirement of the software. It tells the tester exactly what to do and what result to expect, so that the test can be repeated by anyone with the same outcome. If the actual result differs from the expected result, the test case fails and a defect is reported.

### Main components of a test case

- **Test case ID:** a unique identifier (for example TC_LOGIN_001) used for tracking and traceability.
- **Test case title / name:** a short statement of what is being tested.
- **Description / objective:** the purpose of the test case.
- **Module / feature and requirement ID:** the part of the software and the linked requirement, which supports the traceability matrix.
- **Preconditions:** conditions that must be true before execution, for example the user is registered, the browser is open.
- **Test data / input:** the values to be entered, such as username and password.
- **Test steps:** the numbered sequence of actions to be performed.
- **Expected result:** the output or behaviour that the software should show according to the requirement.
- **Actual result:** the output observed during execution.
- **Status:** Pass, Fail, Blocked or Not executed.
- **Postconditions:** the state of the system after the test.
- **Priority and severity:** the importance of the test case.
- **Environment:** operating system, browser, build version.
- **Author, executed by and date, and remarks:** for accountability and notes.

**Example:**

| TC | Steps | Test data | Expected result |
|---|---|---|---|
| TC_LOGIN_001 | 1. Open login page. 2. Enter username and password. 3. Click Login. | User: student01, Password: Abc@1234 | User is logged in and the dashboard is displayed. |

# Question 11

**Explain the process of designing and writing effective test cases.**

## Answer

Designing test cases is a careful activity, because the quality of testing depends on the quality of the test cases. An effective test case is one that has a high chance of finding a defect and is clear, correct and reusable. The process is as follows:

- **1. Understand the requirements:** study the SRS, use cases and user stories, and clear any doubts with the analyst or the customer.

- **2. Identify test scenarios and conditions:** list what is to be tested, including positive and negative situations.
- **3. Select suitable test design techniques:** use equivalence partitioning, boundary value analysis, decision tables and state transition testing to get good coverage with fewer test cases, and error guessing for experience-based cases.
- **4. Prepare test data:** valid, invalid, boundary and special data.
- **5. Write the test cases:** write each with an ID, preconditions, clear and numbered steps, test data and a precise expected result.
- **6. Include positive and negative cases:** check both the correct and the wrong usage.
- **7. Map to requirements:** link each test case to a requirement in the traceability matrix to make sure that all the requirements are covered.
- **8. Review the test cases:** peers and the test lead review them for correctness, completeness and duplication.
- **9. Prioritise and organise:** set high-priority test cases first and group them into suites (smoke, regression).
- **10. Maintain and update:** change the test cases when the requirements change and remove outdated ones.

### Characteristics of effective test cases

- **Clear and simple:** easy to understand and follow by any tester.
- **Accurate:** the expected result is exactly based on the requirement.
- **Independent:** a test case does not depend on the result of another one.
- **Repeatable:** gives the same result every time under the same conditions.
- **Traceable:** linked to the requirement.
- **Economical:** no duplicate test cases; one case checks one thing.
- **Complete in coverage:** covers normal, boundary and error conditions.

## Question 12

**What is Bug Reporting? Explain the important information included in a bug report.**

### Answer

Bug reporting is the process of documenting a defect found during testing and communicating it to the developers and the project team so that it can be understood, reproduced, fixed and verified. A bug report is the written record of the defect. A good report saves time because the developer does not have to ask again and again for details, and it helps the management to track the quality of the product.

### Important information in a bug report

- **Bug ID:** unique number given by the tracking tool.
- **Title / summary:** one-line, clear description of the problem.
- **Description:** detailed explanation of the defect.
- **Steps to reproduce:** exact numbered steps by which the developer can see the defect.
- **Expected result and actual result:** what should happen and what really happened.
- **Environment:** operating system, browser, device, build or version number and test data used.
- **Module / component affected:** the area of the application.
- **Severity:** the technical impact of the bug (critical, major, minor, trivial).
- **Priority:** the urgency of fixing (high, medium, low).
- **Status:** New, Assigned, Open, Fixed, Retest, Verified, Closed, Reopened, Rejected and so on.

- **Reported by / assigned to / date:** the person who found it, the developer responsible and dates.
- **Attachments:** screenshots, screen recording, log files.
- **Reproducibility:** always, sometimes or rarely.
- **Comments / history:** discussion and changes in status.

**Tips for a good bug report:** report one defect per report, use neutral language, give exact steps, check that it is not a duplicate and attach evidence.

# Question 13

**Explain the Bug Life Cycle in software testing.**

## Answer

The bug life cycle (defect life cycle) is the sequence of states through which a defect passes from the time it is found until it is closed. It makes the defect handling process systematic and lets the team know the status of each defect at any moment.

### Stages of the bug life cycle

- **1. New:** the tester finds the defect and reports it for the first time.
- **2. Assigned:** the test lead or project manager reviews the bug and assigns it to a developer.
- **3. Open:** the developer starts to analyse and work on the bug.
- **4. Fixed:** the developer corrects the code and marks the bug as fixed.
- **5. Retest (Pending retest):** the fixed code is given to the tester to verify.
- **6. Verified:** the tester re-tests; if the bug is no longer reproducible it is marked as verified.
- **7. Closed:** after verification the bug is closed.
- **8. Reopened:** if the defect still exists after the fix, the tester reopens it and it goes back to the developer.

### Other possible states

- **Rejected / Not a bug:** the developer or the lead decides that the behaviour is correct as per requirements.
- **Duplicate:** the same defect has already been reported.
- **Deferred / Postponed:** the fixing is moved to a later release because of low priority or time.
- **Cannot reproduce / Need more information:** the developer is unable to see the problem and asks the tester for more details.

**Flow:** New to Assigned to Open to Fixed to Retest to Verified to Closed. If the retest fails, the bug goes from Retest to Reopened and then again to Assigned. If a bug is not valid, it goes from New or Open to Rejected, Duplicate or Deferred.

# Question 14

**What are Test Metrics? Explain their importance in measuring testing activities.**

## Answer

A metric is a measurement of some property of the software or the process. Test metrics are quantitative measures that are collected during testing to give information about the progress, quality and efficiency of the testing activity and the quality of the product. Examples are the number of test cases executed, the number of defects found and the percentage of requirements covered.

### Importance of test metrics

- **Measure progress:** show how much testing is complete and how much is remaining.
- **Measure quality of the product:** the defect density and defect trend show how stable the software is.
- **Decision making:** help management to decide whether the product is ready for release.
- **Measure effectiveness and efficiency of the test team:** for example defects found per hour or defect detection percentage.
- **Process improvement:** show weak areas such as modules with many defects, and help to improve the process.
- **Estimation and planning:** past data helps to estimate effort and time for future projects.
- **Risk identification:** an increasing count of open defects or low coverage warns of problems early.
- **Communication and transparency:** give facts to the stakeholders in the form of reports and graphs.
- **Cost control:** help to measure the cost of quality and the cost of defects.

Metrics must be meaningful, easy to collect, and used for improvement and not for blaming individuals.

# Question 15

**Explain different types of testing metrics used in software testing.**

## Answer

Testing metrics can be classified in different ways. First, by the purpose they serve:

- **Process metrics:** measure the effectiveness of the testing process, for example defect detection efficiency, test case effectiveness and review efficiency.
- **Product metrics:** measure the quality of the software, for example defect density, number of defects by severity and mean time between failures.
- **Project metrics:** measure the progress, cost and schedule of the testing project, for example effort variance and schedule variance.

Second, the commonly used metrics in practice are grouped as follows:

### 1. Test progress / execution metrics

- Number of test cases planned, executed, passed, failed, blocked
- Test execution percentage = (test cases executed / total test cases) x 100
- Pass percentage = (test cases passed / test cases executed) x 100

### 2. Defect metrics

- Total number of defects found, open defects and closed defects
- Defect density = number of defects / size of the software (for example per KLOC or per function point)
- Defect distribution by severity, priority, module and type
- Defect detection percentage (DDP) = defects found in testing / (defects found in testing + defects found after release) x 100
- Defect leakage = defects found after release / total defects x 100
- Defect age: the time between the detection and the closing of a defect

### 3. Coverage metrics

- Requirement coverage = (requirements covered by test cases / total requirements) x 100
- Code coverage: statement, branch and path coverage
- Test case coverage: percentage of the planned tests prepared and executed

### 4. Effort and cost metrics

- Planned effort versus actual effort
- Cost of testing and cost per defect found
- Productivity: test cases written or executed per person per day

### 5. Schedule metrics

- Planned versus actual dates of milestones
- Schedule variance = (actual duration - planned duration) / planned duration x 100

### 6. Quality and effectiveness metrics

- Test effectiveness = defects found by testing / total defects x 100
- Test case effectiveness = defects found by test cases / number of test cases
- Automation coverage and rework (re-opened defects) percentage

# Question 16

**What are Test Case Design Techniques? Explain their importance in software testing.**

## Answer

Test case design techniques are systematic methods used to derive test cases from the requirements, the design or the code. Since exhaustive testing of all inputs is impossible, these techniques help to select a small number of test cases that have the maximum probability of finding defects.

### Categories of techniques

- **Specification-based (black-box) techniques:** test cases are derived from requirements without knowing the code. Examples: equivalence partitioning, boundary value analysis, decision table testing, state transition testing, use case testing and cause-effect graphing.
- **Structure-based (white-box) techniques:** test cases are derived from the internal structure of the code. Examples: statement coverage, branch (decision) coverage, condition coverage and path coverage.
- **Experience-based techniques:** use the knowledge and intuition of the tester. Examples: error guessing, exploratory testing and checklist-based testing.

### Importance of test case design techniques

- **Reduce the number of test cases** while keeping good coverage, which saves time and cost.
- **Increase defect detection:** they focus on the places where defects are likely to occur, such as boundaries.
- **Provide systematic coverage:** no important condition is forgotten.
- **Make testing repeatable and objective:** the result does not depend on the tester's personal choice alone.
- **Help in measuring coverage** and in knowing when to stop testing.
- **Improve quality of test documents** and make review easier.
- **Find requirement defects:** while designing tests, ambiguity and missing rules are discovered.

- **Support maintenance and reuse** of test cases.

# Question 17

**What is Equivalence Partitioning? Explain with a suitable example.**

## Answer

Equivalence Partitioning (EP), also called Equivalence Class Partitioning, is a black-box test design technique in which the input data of the software is divided into groups or classes, called equivalence partitions. All the values in one partition are expected to be processed in the same way by the software. Hence, if one value of a partition detects a defect, the other values of the same partition would also detect it, and if one value works, the others are expected to work too. Therefore only one representative value from each partition is tested.

### Steps

- Identify the input conditions from the requirement.
- Divide each input into valid partitions (acceptable values) and invalid partitions (unacceptable values).
- Select one representative value from each partition.
- Write test cases that cover all valid partitions, and test each invalid partition separately, so that the cause of a failure is clear.

### Example

An application accepts the age of an applicant for a driving licence. The valid age is from 18 to 60 years (both included).

| ID | Partition | Type | Test value | Expected result |
|---|---|---|---|---|
| P1 | Age less than 18 (0 to 17) | Invalid | 10 | Rejected |
| P2 | Age 18 to 60 | Valid | 35 | Accepted |
| P3 | Age more than 60 | Invalid | 75 | Rejected |
| P4 | Non-numeric or blank | Invalid | abc | Error message |

Instead of testing all ages from 0 to 100, only four test cases are used. **Advantages:** fewer test cases, good coverage and suitable for all levels of testing. **Limitation:** it does not give special attention to the edges of the partitions, where defects are common, and for this reason it is used along with boundary value analysis.

# Question 18

**What is Boundary Value Analysis? Explain with a suitable example.**

## Answer

Boundary Value Analysis (BVA) is a black-box test design technique based on the observation that most defects occur at the boundaries (edges) of the input ranges rather than in the centre. Programmers often make mistakes such as using < instead of <=, or an off-by-one error. In BVA the test cases are therefore selected at the minimum and maximum values of a range and just below and just above them.

### Types

- **Two-value BVA:** the boundary value and the value just outside it (for each boundary).
- **Three-value BVA:** the value just below, on and just above each boundary (min-1, min, min+1, max-1, max, max+1). This is the more common type.

### Example

The same age field accepts the age from 18 to 60. The boundary values are 18 and 60.

| TC | Age | Boundary | Expected result |
|---|---|---|---|
| 1 | 17 | Just below minimum | Rejected |
| 2 | 18 | Minimum | Accepted |
| 3 | 19 | Just above minimum | Accepted |
| 4 | 59 | Just below maximum | Accepted |
| 5 | 60 | Maximum | Accepted |
| 6 | 61 | Just above maximum | Rejected |

**Defects found by BVA:** wrong relational operators (age > 18 instead of age >= 18), off-by-one errors, wrong limit constants and incorrect loop counts. **Advantages:** finds many defects with few test cases and is easy to apply. **Limitation:** it works well for ordered numeric ranges but not so well for non-numeric or Boolean data, and it does not consider combinations of inputs.

## Question 19

**Differentiate between Equivalence Partitioning and Boundary Value Analysis.**

### Answer

| Basis | Equivalence Partitioning | Boundary Value Analysis |
|---|---|---|
| Meaning | Divides input data into classes that behave in the same way and tests one value from each class. | Tests the values at and around the edges of the input ranges. |
| Basis of the technique | Assumption that all values of a class are processed equally. | Experience that defects mostly occur at the boundaries. |
| Selected values | Any representative value from the partition (usually from the middle). | Minimum, maximum and values just below and just above them. |
| Focus | Coverage of all partitions (valid and invalid). | Edges between valid and invalid partitions. |
| Number of test cases | Fewer (one per partition). | More (2 or 3 per boundary). |
| Types of defects found | Missing or wrong handling of a whole class of input. | Off-by-one errors, wrong operators (< and <=) and limit errors. |
| Applicability | Numeric ranges, sets of values, formats and types of data. | Mainly ordered data such as numeric ranges, dates and sizes. |
| Relation | Used first to form the partitions. | Extension of EP applied on the edges of the partitions. |
| Example (age 18-60) | Test 10, 35 and 75. | Test 17, 18, 19, 59, 60 and 61. |

**Conclusion:** the two techniques are not alternatives; they are used together to get the best coverage with a small number of test cases.

## Question 20

**What is Test Case Planning? Explain the steps involved in test case planning.**

## Answer

Test case planning is the activity of deciding in advance how the test cases will be created, organised, prioritised and executed for a project. It is a part of test planning and defines which test cases are needed, which techniques will be used to design them, who will write and review them, what the standards and formats are, and how long it will take. Planning the test cases ensures that all the requirements are covered, effort is not wasted on duplicate cases and the important cases are executed first.

### Steps involved in test case planning

- **1. Analyse the requirements and the test plan:** understand the scope, risks and the features to be tested.
- **2. Identify the test conditions and scenarios:** list the situations that must be tested, both positive and negative.
- **3. Decide the test design techniques:** select the techniques (EP, BVA, decision table, state transition, error guessing) suitable for each feature.
- **4. Define the test case format and standards:** fix the template, naming convention, and level of detail.
- **5. Estimate the number of test cases and effort:** estimate how many cases are required and the time for design and review.
- **6. Prioritise the test cases:** rank them on risk and importance so that critical features are tested first when time is limited.
- **7. Plan the test data and the environment:** decide what data is required and how it will be created.
- **8. Assign responsibilities and schedule:** decide who will write, review and execute the test cases and fix the dates.
- **9. Prepare the traceability matrix:** map the requirements to the planned test cases to ensure complete coverage.
- **10. Decide the review and approval process:** the test cases are reviewed by peers and approved.
- **11. Plan maintenance and reuse:** define how the test cases will be updated, grouped into regression suites and automated.